Verilog

```
module seq_detector_1011(
    input clk,
    input rst,
    input din,
    output reg detected
);

reg [2:0] state, next_state;

parameter S0 = 3'b000,
          S1 = 3'b001,
          S2 = 3'b010,
          S3 = 3'b011,
          S4 = 3'b100;

// Present state logic
always @(posedge clk or posedge rst)
begin
    if (rst)
```

```
        state <= S0;
    else
        state <= next_state;
end

// Next state logic
always @(*) begin
    case(state)
        S0: next_state = din ? S1 : S0;
        S1: next_state = din ? S1 : S2;
        S2: next_state = din ? S3 : S0;
        S3: next_state = din ? S4 : S2;
        S4: next_state = din ? S1 : S2;
        default: next_state = S0;
    endcase
end

// Output logic
always @(*) begin
    detected = (state == S4);
end
```

```
endmodule
```

Testbench

```
module tb_seq_detector_1011;

reg clk;
reg rst;
reg din;
wire detected;

seq_detector_1011 uut (
    .clk(clk),
    .rst(rst),
    .din(din),
    .detected(detected)
);

// Clock generation
always #5 clk = ~clk;
```

```
initial begin
    clk = 0;
    rst = 1;
    din = 0;

    #10 rst = 0;

    // Input sequence: 1 0 1 1
    #10 din = 1;
    #10 din = 0;
    #10 din = 1;
    #10 din = 1;

    #20;

    // Another sequence
    #10 din = 1;
    #10 din = 0;
    #10 din = 1;
    #10 din = 1;
```

```
    #30 $finish;
end

initial begin
    $monitor("Time=%0t rst=%b din=%b
detected=%b",
             $time, rst, din, detected);
end

endmodule
```

Synthesis

```
create_clock -name clk -period 10
[get_ports clk]
set_input_delay 2 -clock clk [get_ports din]
set_output_delay 2 -clock clk [get_ports
detected]
set_false_path -from [get_ports rst]
```